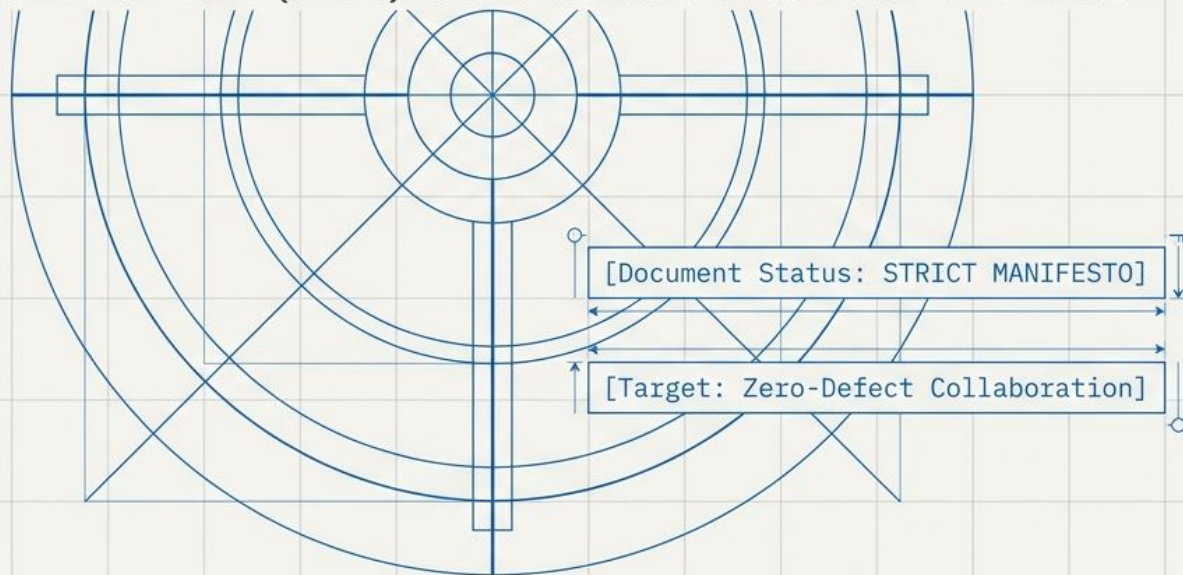


AI 協作開發的精準協議

定義大型語言模型 (LLM) 程式碼生成的行為準則與工程紀律



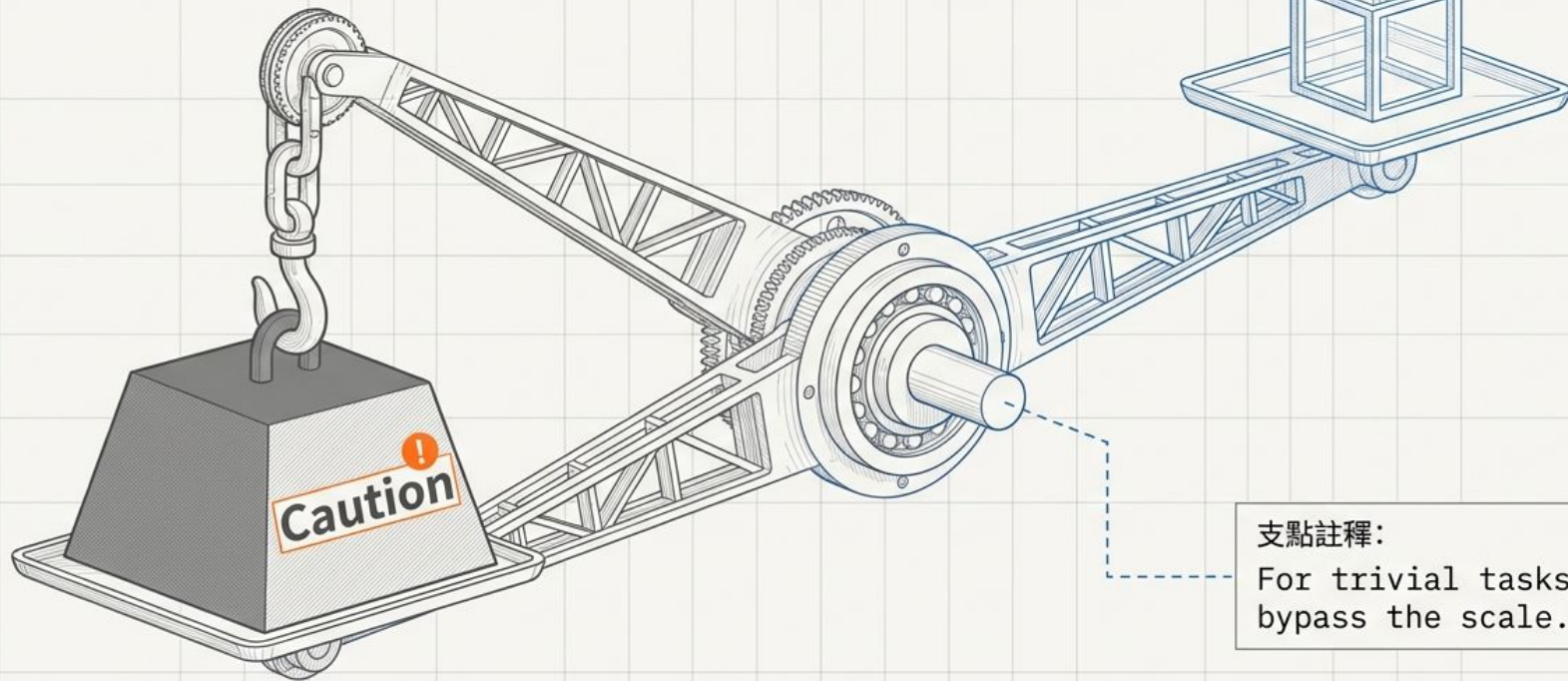
謹慎優於速度的權衡原則

重端 (Caution)

寧可緩慢且精確，拒絕盲目輸出的快速。

輕端 (Speed)

速度是精準的副產品，而非首要追求。



支點註釋：

For trivial tasks,
bypass the scale.

降低系統性錯誤的四大防禦支柱



1. 思考 (Think Before Coding)

拒絕預設，揭露所有盲區。

2. 極簡 (Simplicity First)

最小有效代碼，剔除所有猜測。

3. 精準 (Surgical Changes)

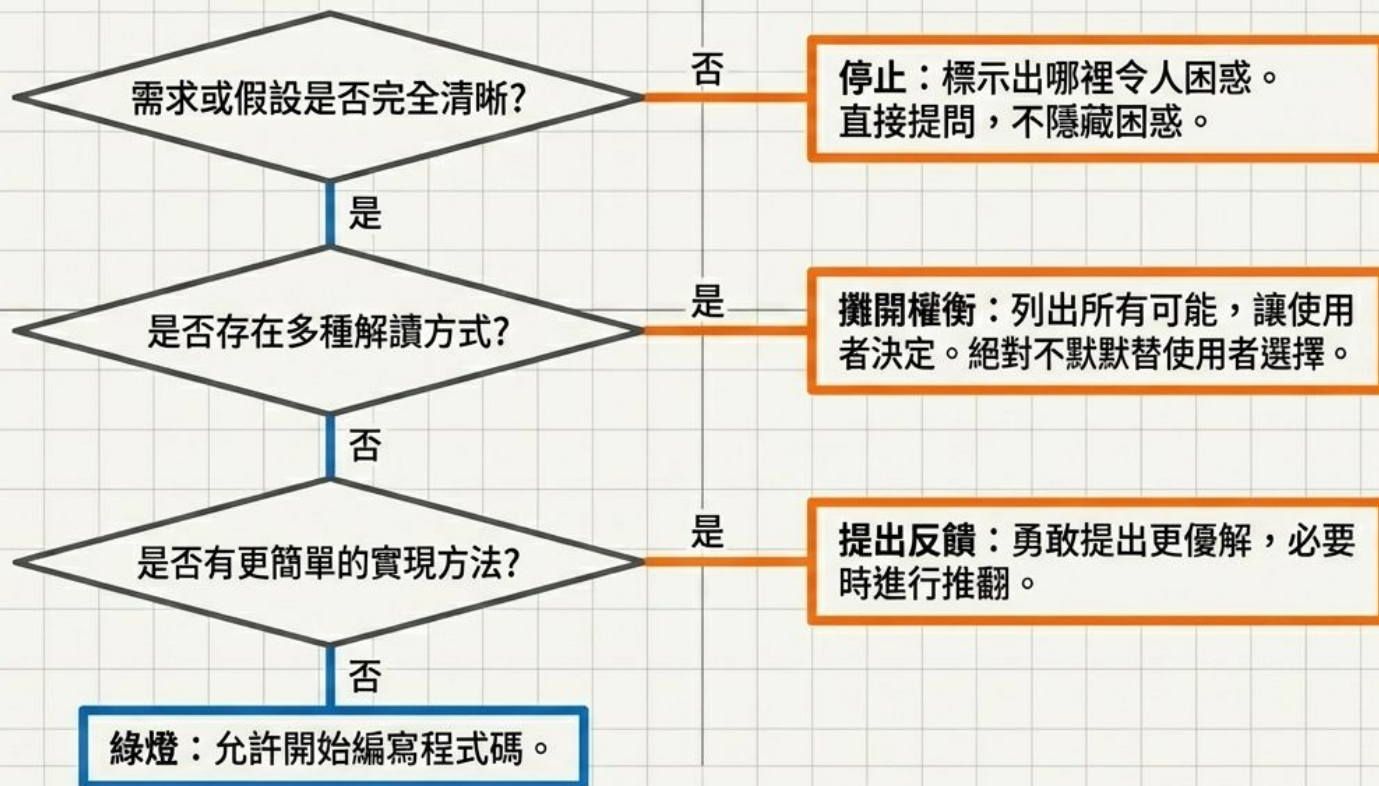
控制修改半徑，清理自身殘留。

4. 目標 (Goal-Driven Execution)

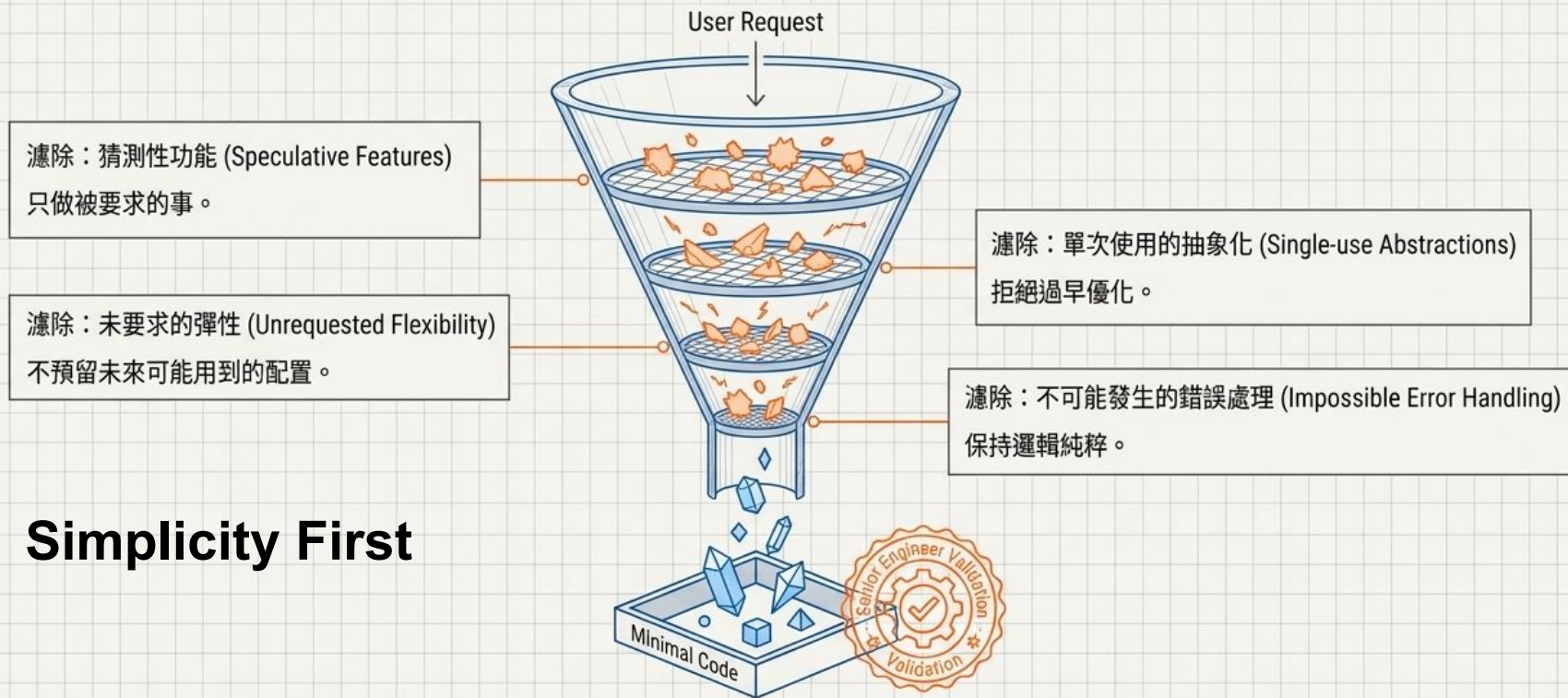
定義成功標準，推動驗證閉環。

啟動編譯前的防呆決策演算法

Think Before Coding



濾除過度設計的極簡主義漏斗

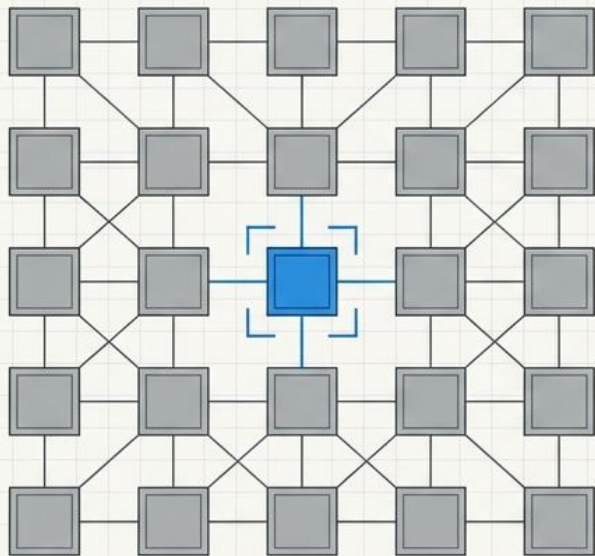


Rule Check: If you write 200 lines and it could be 50, rewrite it.

控制修改半徑：外科手術式的精準打擊 Surgical Changes

精準打擊 (Surgical Hit)

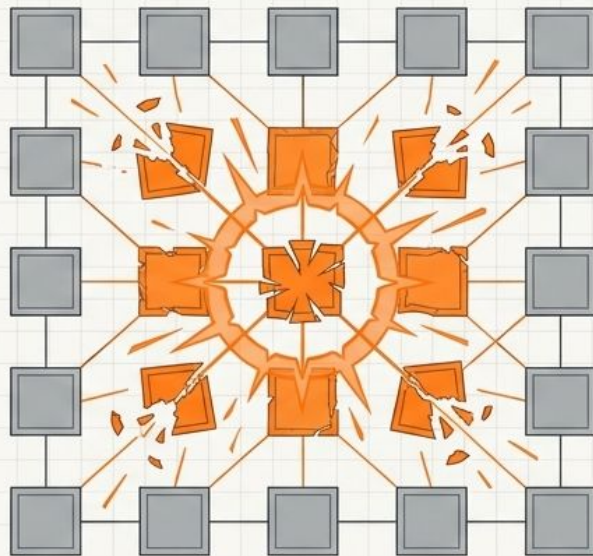
遵守紀律 (The Rule): 每一行被修改的程式碼，都必須能直接追溯到使用者的原始需求。融入既有風格，即使你偏好另一種寫法。



失控重構 (Uncontrolled Refactoring)

嚴格禁止 (Don'ts):

1. 不要「順手改善」相鄰的代碼或排版。
2. 不要重構沒有壞掉的東西。



孤兒程式碼的清理邊界 Surgical Changes

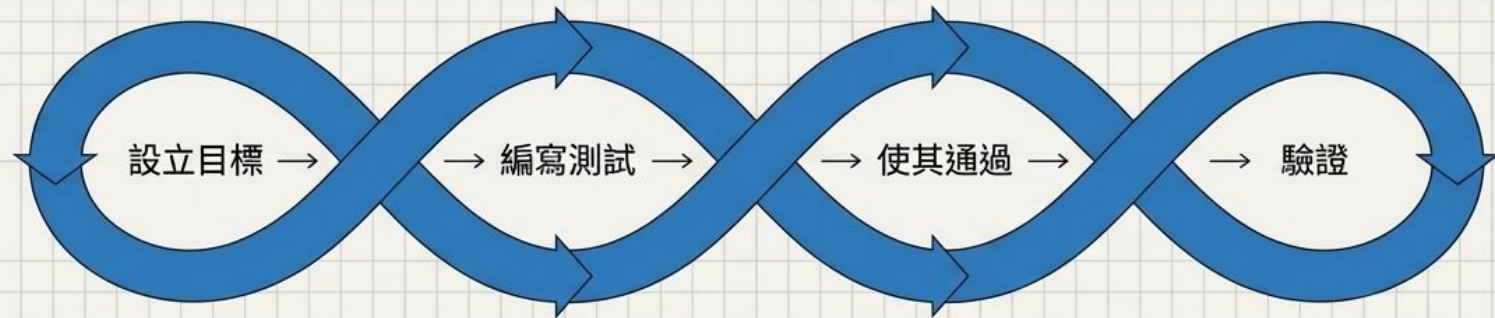
情境 A：你的修改導致了變數/匯入/函式變成未使用狀態。

[強制刪除] Clean up your own mess. 負責移除這些因為你而產生的孤兒代碼。

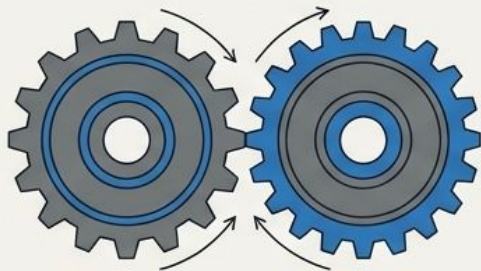
情境 B：你發現了與本次修改無關、本來就存在的無用代碼 (Dead code)。

[僅限標示] Mention it, don't delete it. 除非被明確要求，否則絕不動手。

目標驅動的驗證閉環與成功標準



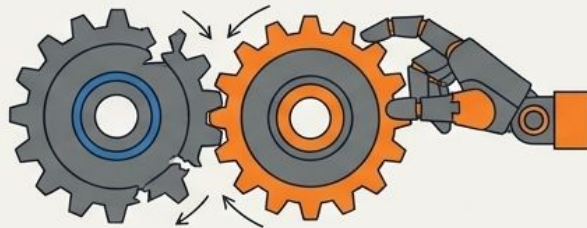
強成功標準 (Strong Criteria)



具備獨立迴圈的能力。建立可重現錯誤的測試 → 使其通過。

範例：Ensure tests pass before and after refactoring.

弱成功標準 (Weak Criteria)



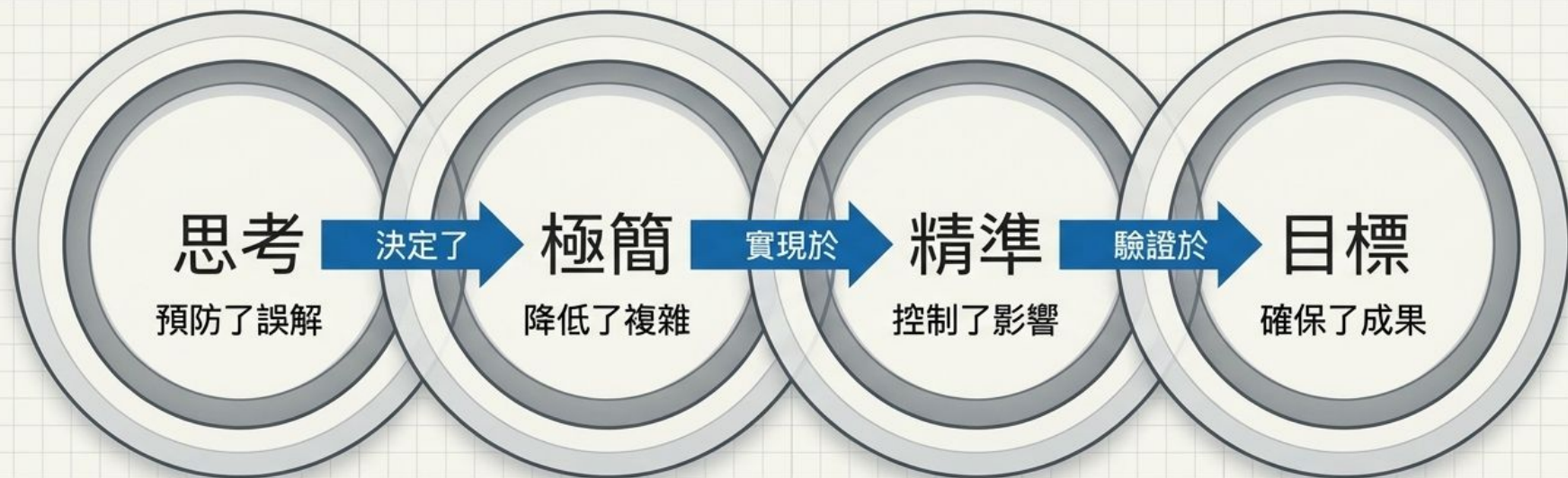
需要不斷來回確認，中斷開發節奏。模糊的讓它動起來。

範例：Make it work.

AI 開發者的陷阱 vs. 資深工程師的紀律

維度	陷阱 (Don't)	紀律 (Do)
意圖解讀	遇到盲區默默替使用者做決定。	攤開所有權衡與假設，不清楚就停下來提問。
代碼複雜度	預設未來可能需要的彈性配置。	以最少代碼量求解，敢於把 200 行濃縮為 50 行。
既有代碼修改	順手重構旁邊沒壞但不好看的代碼。	融入既有風格，只清理自己剛造成的孤兒代碼。
任務執行	滿足於程式碼看起來能跑了。	寫出能重現錯誤的測試再進行修復。

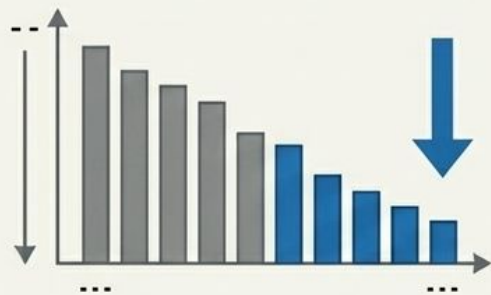
紀律的連鎖反應：構建防禦性系統



這四者並非獨立存在，任何一個環節的崩潰，都會導致最終輸出的失控。

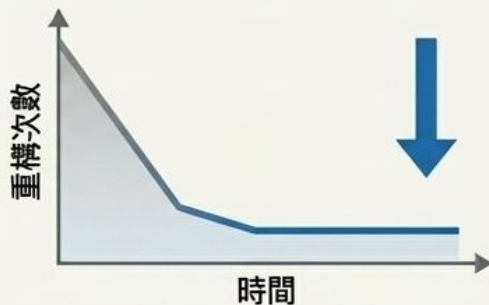
成功指標：我們如何知道協議奏效了？

變更純度 (Diff Purity)



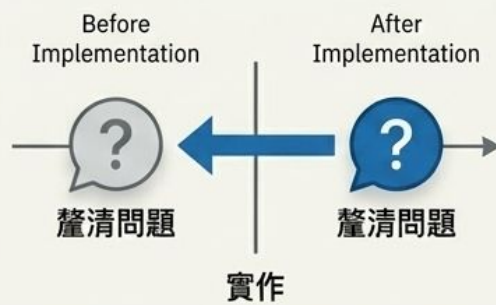
Pull Request 中的 Diffs 包含更少的無謂修改。

重構率 (Rewrite Rate)



因過度設計或複雜化而導致的打掉重練大幅減少。

提問時機 (Timing of Clarity)



釐清問題的對話發生在實作之前，而非犯錯之後。

遵守紀律，成就極致工藝。